

CLI, MAVEN & DEV MODE

Command	Purpose
quarkus create app g:a	scaffold project
quarkus dev	dev mode, live reload
quarkus ext add <name>	add extension
quarkus build	package (fast-jar)
quarkus build --native	native executable
quarkus image build	build container image
quarkus deploy k8s	generate+apply manifests
quarkus update	migrate platform version

Maven equiv.: `./mvnw quarkus:dev`, `quarkus:add-extension -Dextensions=...`, `package -Dnative`, `quarkus:deploy -Dquarkus.kubernetes.deploy=true`

Dev-mode keys: r=resume s=run tests f=force-run d=Dev UI h=help o=toggle output q=quit

PROJECT LAYOUT

```
src/main/java/...
app code
src/main/resources/
application.properties
META-INF/resources/
static
templates/
db/migration/
Flyway
src/test/java/...
@QuarkusTest
src/test/resources/
application.properties
test overrides
pom.xml | build.gradle[.kts]
```

CDI (ARC) ANNOTATIONS

Scopes: `@ApplicationScoped` (default sgl.), `@Singleton` (no proxy), `@RequestScoped`, `@Dependent` (new per inj. pt.), `@SessionScoped` (caution, needs session support)

Producers: `@Produces` / `@Disposes` on a method; **Qualifiers:** custom `@Qualifier` annotation, `@Named`, `@Alternative` + `@Priority`, `@Default`

Interceptors: `@InterceptorBinding` on the annotation, `@Interceptor` + `@AroundInvoke` on the impl., `@Priority` orders them; decorators via `@Decorator`

Lifecycle: void `onStart(@Observes StartupEvent e)`, `@Observes ShutdownEvent`, `@PostConstruct` / `@PreDestroy` on bean methods

CONFIGURATION

Precedence (high→low): system props > env vars > `.env (dev)` > `%profile.*` in application.properties > application.properties > application.yml > microprofile-config.properties > `@ConfigMapping/@ConfigProperty` defaults

Profiles: `%dev.quarkus.log.level=DEBUG`; activate: `-Dquarkus.profile=stage` or `QUARKUS_PROFILE`; built-in `dev/test/prod`

```
@ConfigMapping(prefix = "app")
public interface AppConfig {
    String name();
    Optional<Integer>
    retries();
    Nested nested();
    interface Nested { String
    value(); }
}
```

Build-time-fixed props (e.g. `quarkus.datasource.db-kind`) are baked into the artifact/native image — changing them after build fails fast; others are read at runtime startup.

QUARKUS REST

```
@Path @GET @POST @PUT @DELETE
@PATCH · @PathParam @QueryParam
@HeaderParam @RestPath @RestQuery
@RestHeader ·
@Produces/@Consumes(MediaType...) ·
@Blocking/@NonBlocking

@Path("/")hello")
public class HelloResource {
    @GET

    @Produces(MediaType.TEXT_PLAI
    N)
    public String hello() {
        return "hello"; }

    @POST @Blocking
    public Response
    create(@Valid Item i) {
        return
        Response.status(CREATED).enti
        ty(i).build();
    }
}
```

REST CLIENT

```
@RegisterRestClient(configKey
= "my-api")
public interface MyClient {
    @GET @Path("/items/{id}")
    Uni<Item>
    get(@PathParam("id") long
    id);
}

quarkus.rest-client.my-
api.url=https://api.example.co
m

Inject: @RestClient MyClient
client; · headers:
@ClientHeaderParam · errors:
ResponseExceptionMapper
```

PANACHE & @TRANSACTIONAL

Active record: class `Book` extends `PanacheEntity` { public String title; } → `Book.listAll()`, `Book.find("title", t).firstResult()`, `book.persist()`

Repository: interface `BookRepo` extends `PanacheRepository<Book>{}` inject & call `repo.listAll()`

```
.find("title query+sort
",Sort.by("t
itle"))

.page(Page.o pagination
f(0,10)).lis
t()

.find("#Book named query
.byAuthor",P
arameters.wi
th("a",v))

.count() / count/delete
.deleteAll()
```

@Transactional: on write-path service methods; rolls back on unchecked exceptions by default; tune with `rollbackOn / dontRollbackOn`; TxType propagation values (REQUIRED default, REQUIRES_NEW, MANDATORY, SUPPORTS, NOT_SUPPORTED, NEVER)

MUTINY UNI / MULTI

Type	Emits
Uni<T>	0..1 item (or failure)
Multi<T>	0..N items (stream)

Lazy, cold until `.subscribe()`:

```
.onItem().transform(x->..).
.onItem().transformToUni/Multi
i(..) (chain),
.combine().all().unis(a,b),
.merge().concatenate(),
.onFailure().recoverWithItem(
..),
.onFailure().retry().atMost(n
), .ifNoItem().after(d).fail()
(timeout)
.subscribe().with(item -> ..,
failure -> ..)
```

@RUNONVIRTUALTHREAD

Runs a REST/gRPC/messaging method's blocking body on a JDK virtual thread instead of the platform worker pool — write plain blocking code, still get high concurrency.

Avoid `synchronized` (pins the carrier thread); watch `ThreadLocal` cost; needs Java 21+ (the Quarkus 4 baseline). Benchmark on your own workload.

REACTIVE MESSAGING

```
@Incoming("orders")
@Outgoing("processed")
Order process(Order o) {
    return enrich(o); }

@Channel("processed")
Emitter<Order> emitter;
emitter.send(order);
// or MutinyEmitter

Ack: connector default, or
@Acknowledgment(Strategy.MANU
AL) + Message.ack()/nack(t);
Kafka DLQ via ..failure=
strategy=dead-letter-queue
```

FAULT TOLERANCE (MP)

Annotation	Default
@Retry	maxRetries=3, delay=0
@Timeout	1000 ms
@CircuitBreaker	reqVolThreshold=20, failRatio=0.5, delay=5000ms
@Bulkhead	value=10 concurrent
@Fallback	fallbackMethod=..
@Asynchronous	CompletionStage/Uni return

SECURITY & OIDC

```
@RolesAllowed("admin") @Authenticated
@PermitAll @DenyAll
@PermissionsAllowed("read")

quarkus.http.auth.permission.
pub.paths=/public/*
quarkus.http.auth.permission.
pub.policy=permit
quarkus.http.auth.permission.
adm.paths=/admin/*
quarkus.http.auth.permission.
adm.policy=role-adm
quarkus.http.auth.policy.role
-adm.roles-allowed=admin

OIDC web-app: application-
type=web-app + auth-code flow,
session cookie · service:
application-type=service +
bearer-token validation via JWKS. Both
need quarkus.oidc.auth-server-
url=..
```

/Q/* ENDPOINTS

/q/health	liveness+readiness+st artup
/q/health/live	liveness only
/q/health/read	readiness only
/q/metrics	Micrometer/Promethe us
/q/openapi	OpenAPI document
/q/swagger-ui	Swagger UI
/q/dev	Dev UI (dev mode only)
/q/info	build/git/OS info

TESTING

@QuarkusTest	boots the CDI app for the test
@QuarkusIntegrationTest	runs the packaged/native artifact
@TestProfile	swap config for one test class
@TestSecurity("alice", roles="admin")	fake identity
@InjectMock / @InjectSpy	Mockito bean replacement

DEV SERVICES (SUPPORT)

Postgres/MySQL/MariaDB/DB2/MSSQL/ Oracle (JDBC & reactive), Kafka, Redis, MongoDB, Keycloak, Elasticsearch, Infinispan, AMQP (RabbitMQ), Vault, LGTM (observability) — auto-start a container when no URL is configured; reuse across runs with `quarkus.devservices.reusable=true`; disable per-service with `..devservices.enabled=false`

PACKAGING & NATIVE

quarkus.package.jar.type	fast-jar (default) uber-jar mutable-jar legacy-jar
quarkus.native.container-build	build in a builder container
quarkus.native.builder-image	Mandrel/GraalVM image
quarkus.native.resources.includes	extra resources for the image

CONTAINER IMAGE & KUBERNETES

quarkus.container-builder.image.build	build the image on package
quarkus.container-builder.image.build	jib docker buildpack
..registry/group/name/tag	image coordinates
quarkus.kubernetes.deployment-target	kubernetes knative openshift
quarkus.kubernetes.replicas	replica count

QUTE TEMPLATING

```
{name} expression
{#if cond}.. conditional
{#else}..
{/if}

{#for item loop
in items}..
{/for}

{#let x=1}.. local binding
{/let}

Type-safe: @CheckedTemplate static
methods matching template names,
validated at build time.
```

SPRING → QUARKUS

@RestController	@Path resource
@Service/@Component	@ApplicationScoped bean
@Autowired	@Inject
@Value	@ConfigProperty/@C onfigMapping
@Scheduled	@Scheduled (quarkus-scheduler)
Spring Data JPA repo	Panache repository
application.yml	application.properties /yml